

Toegepaste Informatica

3TX

2025-2026



UCLL
HOGESCHOOL



Full-Stack Software Development

Back-end: New Java features

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Some new Java features

- In this ppt:
 - The var keyword
 - Working with records
 - Working with enums

The var keyword

- The var keyword allows a variable to be initialized without having to explicitly declare its type.

```
Pony pony = ponyRepository.findPonyByName(name);
```

```
var pony = ponyRepository.findPonyByName(name);
```

- The compiler uses **Type inference** to detect the datatype of a variable
- var can only be used if the type can be **unambiguously** inferred from the right-hand side of the equal sign

where can I use var

- Local variables

```
public void exampleVarDeclaration {  
    var a = 10;  
    var b = 30;  
    System.out.println (a + b);  
}
```

where can I use var

- In for loops:

```
1  public void exampleForDeclaration {  
2      for (var i = 0; i < 10; i++) {  
3          ...  
4      }  
5  }
```

where can I use var

- In try-with-resource:

```
1 public void exampleTryDeclaration {  
2     try (var in = new FileInputStream (path)) {  
3         ...  
4     }  
5 }
```

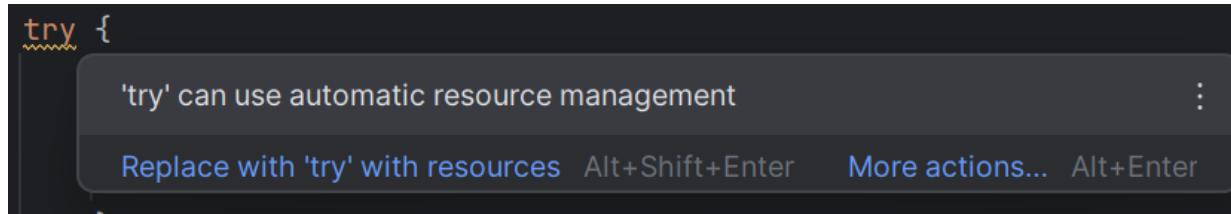
Try with resources?

- The classic way of writing a try-catch-finally that uses a resource like scanner:

```
public static void main(String[] args) {  
    Scanner scanner = null;  
    try {  
        scanner = new Scanner(new File("test.txt"));  
        while (scanner.hasNext()) {  
            System.out.println(scanner.nextLine());  
        }  
    } catch (  
        FileNotFoundException e) {  
        e.printStackTrace();  
    } finally {  
        if (scanner != null) {  
            scanner.close();  
        }  
    }  
}
```

Try with resources?

- IntelliJ will tell you that there is a better way when you hover over the try keyword



- And suggest this code instead:

```
public static void main(String[] args) {  
    try (Scanner scanner = new Scanner(new File("test.txt"))) {  
        while (scanner.hasNext()) {  
            System.out.println(scanner.nextLine());  
        }  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```

Records

- Records are **immutable data classes** that require only the type and name of fields.
- The *equals*, *hashCode*, and *toString* methods, as well as the *private*, *final* fields and *public* constructor, are generated by the Java compiler.

```
public record Person(String name, String address) {}
```

- Records can't inherit from other classes, because they already implicitly inherit from the `java.lang.Record` class.

Records: if we would write it ourselves

```
public class Person { 3 usages

    private final String name; 6 usages
    private final String address; 5 usages

    public Person(String name, String address) { no usages
        this.name = name;
        this.address = address;
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, address);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        } else if (!(obj instanceof Person)) {
            return false;
        } else {
            Person other = (Person) obj;
            return Objects.equals(name, other.name)
                && Objects.equals(address, other.address);
        }
    }

    @Override
    public String toString() {
        return "Person [name=" + name + ", address=" + address + "];"
    }

    public String getName() { no usages
        return name;
    }
}
```

- Final field for each data member
- Getter for each field
- Public constructor
- Equals method
- hashCode method
- toString method



All of this, just for a simple immutable data object, with 2 strings as data pieces?

Immutability

- What?
 - Immutability in refers to **objects whose state cannot change after they are created.**
- Examples:
 - Strings in Java are immutable, when you change the value, a completely new object is created.
 - Any field marked final

records: constructor

- **Canonical Constructor:** automatically created for you

```
public record Person(String name, int age) {  
    // autogenerated code  
    // private final String name;  
    // private final int age;  
    // public Person(String name, int age) {  
    //     this.name = name;  
    //     this.age = age;  
    // }  
}
```

records: constructor

- **Compact Constructor:** add logic to the canonical constructor
 - There is no need to declare the constructor arguments

```
public record Person(String name, int age) { no usages
    public Person { no usages
        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }
    }
}
```

records: overriding methods

- In case you don't like the implementation of one of the automatically created methods, you can still override them

```
public record Person(String name, int age) { no usages
    @Override
    public String toString() {
        return String.format("Person{name='%s', age=%d}", name, age);
    }
}
```

records: adding methods

- Records can have their own methods. (remember, they can't change the state of the record)

```
public record Person(String name, int age) {  
    public boolean isAdult() { no usages  
        return age >= 18;  
    }  
}
```

- Note: this also means that records can implement Interfaces

Enums

- An enum is a special "class" that represents a group of **constants**
 - Short for enumeration

```
public enum OrderStatus { no usages new *  
    PENDING, SHIPPED, DELIVERED; no usages  
}
```

```
enum WorkflowStatus { no usages  
    NOT_STARTED, IN_PROGRESS, COMPLETED, FAILED;  
}
```

- In java an enum can have methods:

```
public enum OrderStatus { no usage  
    PENDING, SHIPPED, DELIVERED;  
    public boolean isShipped() {  
        return this == SHIPPED;  
    }  
}
```

```
enum Day { no usages  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;  
  
    public boolean isWeekend() { no usages  
        return this == SUNDAY || this == SATURDAY;  
    }  
}
```

Enums: why

- only allow a fixed list of values
 - The IDE will present those values in the autocomplete
- Readability:
 - It's easier to read: `WeekDay.Monday` than it is to read `WeekDay.0`
 - Is that starting at Sunday or on a Monday?

Enums: in a switch statement

```
enum Day { 2 usages
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, 1 usage
    THURSDAY, FRIDAY, SATURDAY no usages
}

public class EnumTest { no usages
    Day day; 2 usages

    public EnumTest(Day day) { no usages
        this.day = day;
    }

    public void tellItLikeItIs() { no usages

        switch (day) {
            case MONDAY:
                System.out.println("Mondays are bad.");
                break;

            case FRIDAY:
                System.out.println("Fridays are better.");
                break;

            case SATURDAY:
            case SUNDAY:
                System.out.println("Weekends are best.");
                break;

            default:
                System.out.println("Midweek days are so-so.");
                break;
        }
    }
}
```

- IntelliJ can automatically create all branches for you:

```
switch (day) {
    case SUNDAY -> {
    }
    case MONDAY -> {
    }
    case TUESDAY -> {
    }
    case WEDNESDAY -> {
    }
    case THURSDAY -> {
    }
    case FRIDAY -> {
    }
    case SATURDAY -> {
    }
}
```

Enums: how to persist to a db

- The easiest way to persist an enum to a database is to store it in a text column (TEXT or VARCHAR)
- And use the `@Enumerated(EnumType.STRING)` annotation

```
CREATE TABLE "user" (  
    id BIGSERIAL NOT NULL,  
    -- snipped  
    role TEXT NOT NULL,  
  
    CONSTRAINT user_pkey PRIMARY KEY (id)  
);
```

```
public class User {  
    @Enumerated(EnumType.STRING)  
    @NotNull  
    private Role role;  
}
```